

MARWADI UNIVERSITY

DEPARTMENT OF AI, ML

DAA (01AI0506) — Lab Manual

Name: Eluri sravan Enrollment No: 92400118905

PRACTICAL - 6

Implementation of Matrix Chain Multiplication using Dynamic Programming

Aim of the Practical:

To find the parenthesization with minimum scalar multiplications using dynamic programming and derive its complexity.

Algorithm:

- 1 Initialize the DP table with 0 for single matrices.
- 2 Consider chain lengths from 2 to n.
- 3 For each matrix chain, try every possible split point k.
- 4 Calculate the multiplication cost for each split.
- 5 Store the minimum cost in the DP table.
- 6 Return dp[1][n] as the minimum number of scalar multiplications.

Program:

```
#include <iostream>
#include <vector>
#include <climits>
using namespace std;

long long matrixChain(const vector<int>& p) {
    int n = p.size() - 1;
    vector<vector<long long>> dp(n + 1, vector<long long>(n + 1, 0));

    for (int len = 2; len <= n; len++) {
        for (int i = 1; i <= n - len + 1; i++) {
            int j = i + len - 1;
            dp[i][j] = LLONG_MAX;

            for (int k = i; k < j; k++) {
                long long cost = dp[i][k] + dp[k + 1][j]
                    + 1LL * p[i - 1] * p[k] * p[j];

                if (cost < dp[i][j])
                    dp[i][j] = cost;
            }
        }
    }

    return dp[1][n];
}

int main() {
    vector<int> p = {4, 10, 3, 8};
    cout << matrixChain(p);
}
```

Output:

Step-by-Step Analysis:

- $A1 = 4 \times 10$, $A2 = 10 \times 3$, $A3 = 3 \times 8$.
- $(A1A2)A3$: cost = $(4 \times 10 \times 3) + (4 \times 3 \times 8) = 120 + 96 = 216$.
- $A1(A2A3)$: cost = $(10 \times 3 \times 8) + (4 \times 10 \times 8) = 240 + 320 = 560$.
- Minimum = 216 scalar multiplications.

Time Analysis:

- Time Complexity: $\Theta(n^3)$.
- Space Complexity: $\Theta(n^2)$.

Result:

Matrix Chain Multiplication was solved successfully. The sample minimum cost is 216 and the DP complexity is cubic in the number of matrices.